UNIVERSITY OF SCIENCE – VNU-HCM

FACULTY OF INFORMATION TECHNOLOGY

----------∞O∞----------

Final Project Report

ChineseNom Translator (Machine Translation via mT5)

Instructors: Mr. Ngô Minh Nhựt

Mr. Lê Long Quốc

Students: 22127020 – Phan Lê Đức Anh

22127233 – Trần Hoàng Linh

22127421 – Nguyễn Thị Ngọc Trang

22127482 – Nguyễn Gia Phúc

Ho Chi Minh, 24th August, 2025

Acknowledgements

We would like to express our sincere gratitude to the lecturers who guided us throughout the completion of this project.

We are deeply indebted to Mr. Ngô Minh Nhựt for his patient and thorough supervision over the past period.

We also gratefully acknowledge the quiet support and encouragement from Mr. Ngô Minh Nhựt and Mr. Lê Long Quốc during the implementation process.

The valuable knowledge that Mr. Ngô Minh Nhựt and Mr. Lê Long Quốc imparted in class-together with the detailed guidance provided through homework assignments-gave us a solid foundation to complete this project.

Although the process was demanding and filled with challenges, our team persevered and achieved the final result. This accomplishment is owed in no small part to the assistance of our lecturers.

Contents

Acknowledgements 1

I. Introduction 3

II. Problem Statement & Objectives 3

III. Project Overview 3

IV. System Architecture & Technologies 3

V. Dataset 4

1. Sources and coverage 4
2. Known limitations 4
3. Analysis on Split Datasets 5
4. Preprocessing 9

VI. Model & Training 9

1. Model Introduction 9
2. Model Architecture 10
3. Training 13
4. Training Results 14

VII. Evaluation 14

1. Metric 14
2. Evaluation Result 15

VIII. Web Application 15

IX. Environment Setup & Usage 15

X. Limitations 16

XI. Future Work 16

XII. References 16

I. Introduction

Transformer architectures have become the standard for Natural Language Processing (NLP), enabling strong results across tasks such as translation, summarization, and question answering. This project builds a practical application on top of the Transformer family to translate text written in Chinese/Hán–Nôm–style input into modern Vietnames. The work satisfies the course requirement to select an NLP problem and retrain a Transformer-based model with an evaluation, and provide a working web application for end users.

II. Problem Statement & Objectives

Students and self-learners working with Hán–Nôm/Chinese historical materials often need Vietnamese translations quickly and consistently. Manual translation is time-consuming and inconsistent; general LLMs are not always domain-adapted to historical/orthographic variants.

Objectives:

1. Build a domain-adapted MT model using mT5.
2. Evaluate with standard automatic metrics and provide qualitative examples.
3. Deliver an end-to-end web app so non-technical users can translate documents or sentences.

III. Project Overview

Goal: Automatic translation from Chinese/Hán–Nôm (source) to Vietnamese (target) using a fine-tuned multilingual T5 (mT5) model; provide a simple web UI and API for interactive translation.

Core idea: Start from a pretrained sequence-to-sequence Transformer, fine-tune it on a parallel corpus, and expose the model through a lightweight web server.

Key Features:

- Batch and single-sentence translation using an mT5-based encoder–decoder.
- Robust preprocessing: Unicode normalization, punctuation cleanup, length filtering, and deduplication.
- Reproducible training scripts and evaluation with standard MT metrics (sacreBLEU).
- Web application for interactive use.

Intended Users / Beneficiaries:

- **Students & self-learners** studying East Asian texts who need Vietnamese translations.
- **Language learners** who want bilingual sentence pairs for vocabulary acquisition.
- **Educators & researchers** who need a baseline MT system for Hán–Nôm resources.
- **Professionals** preparing materials that include classical/Chinese text with Vietnamese counterparts.

IV. System Architecture & Technologies

Modeling Stack

- **Transformers (Hugging Face)** with `MT5ForConditionalGeneration`.

- **SentencePiece** tokenizer (mT5 default).
- **PyTorch** for training/inference.
- **sacreBLEU** and for evaluation.

Application Stack

- **Streamlit** UI server.
- **Python 3.11** environment with `pip` or `pipenv`.

V. Dataset

Supervised machine translation with a parallel corpus **Chinese/Hán–Nôm → Vietnamese**.

(The IHR-Nom Project)

1. Sources and coverage

Truyện Kiều (Tale of Kiều). VNPF hosts multiple historical editions and variants (e.g., 1866–1904), supplying Nôm text with accompanying quốc ngữ lines and page images. These parallel lines allow reliable alignment without external heuristics.

Lục Vân Tiên. Digitized from the National Library of Vietnam/VNPF partnership, with Nôm lines presented alongside Vietnamese text ("Trước đèn xem chuyện Tây Minh…")—again enabling clean, line-level pairing.

Chinh Phụ Ngâm Khúc. VNPF provides introductions and full text, including Nôm renderings of the originally Hán work; representative pages show interleaved Nôm and Vietnamese lines suitable for parallel extraction.

Thơ Hồ Xuân Hương. The collection "Poems of Hồ Xuân Hương" (with English/Vietnamese scaffolding across items) supplies standardized Nôm text and metadata. Selected poems include line-parallel structure usable for alignment.

Lịch sử Việt Nam (History of Greater Vietnam / Đại Việt sử ký toàn thư). VNPF hosts a searchable full text; we extract chapters that include Nôm content and Vietnamese renderings when present, treating them as semi-parallel material (longer segments, paragraph alignment).

ChuNom.org Corpus Shelf. A machine-learning oriented corpus "bookshelf" containing dozens of Nôm documents; we incorporate documents that provide or can be paired with quốc ngữ counterparts.

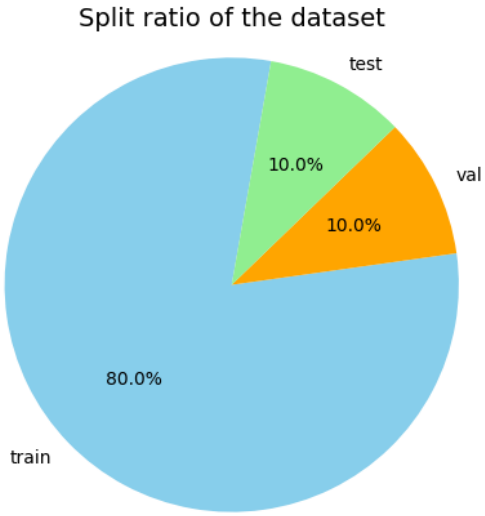
Nôm dictionary for normalization (reference only, not parallel text). The digital "Tự Điển Chữ Nôm Dẫn Giải" provides Unicode code points, readings, structural notes, and quotations from many Nôm works. We use it to standardize characters, resolve variants, and assist tokenization—not as parallel supervision.

2. Known limitations

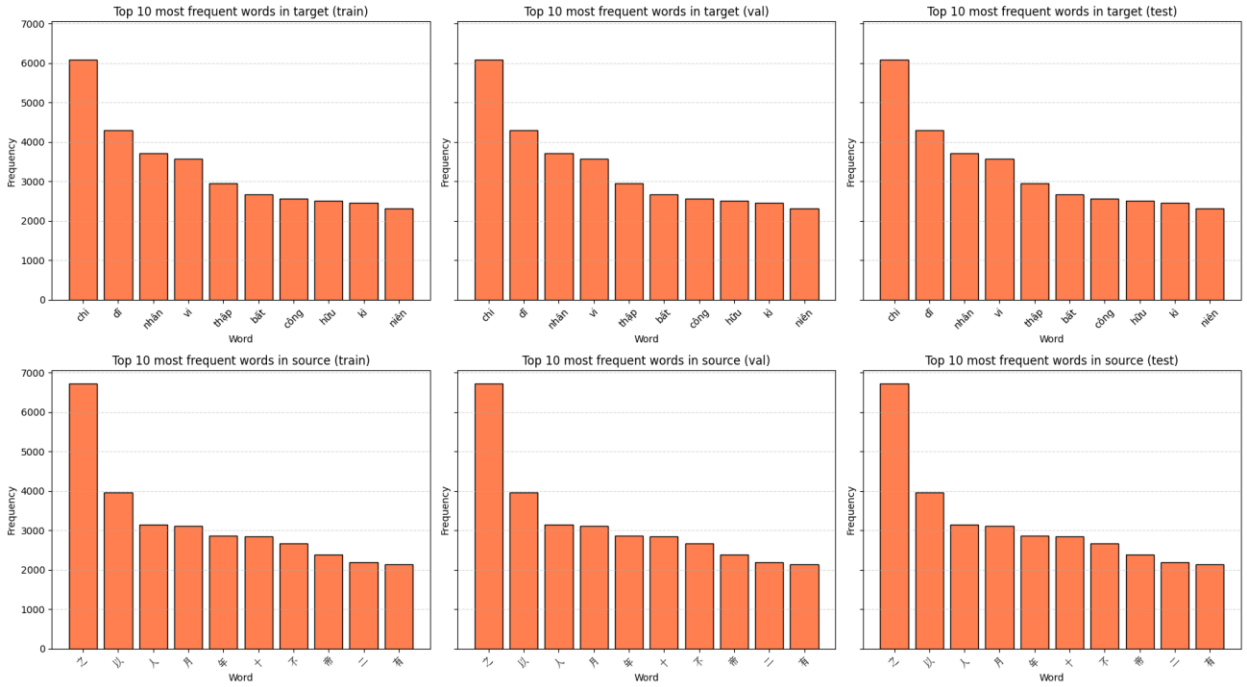
Historical orthography and mixed Hán–Nôm registers lead to character-variant noise; some collections (e.g., historical chronicles) lack strict line-parallel Vietnamese, requiring heuristic alignment and manual spot-checks. Poetry often aligns neatly line-by-line; prose may need paragraph-level alignment. The dictionary helps normalize variants but does not eliminate ambiguity.

3. Analysis on Split Datasets

Split ratio:



Top 10 most frequent tokens in each of the split datasets



The figure shows the top-10 most frequent tokens for both the target (Vietnamese) and the source (Hán–Nôm/Chinese) in the train, validation, and test splits. The bars show raw counts.

Target (Vietnamese) — top rows

Across train/val/test, the most frequent words are stable and dominated by function words/particles and high-frequency Sino-Vietnamese items (e.g., “chi, dĩ, nhân, vi, thập …”), with small changes in ranking between splits. This stability indicates that (i) the splits preserve the same register and domain, and (ii) there is no obvious distribution shift between training and held-out data. The steep

drop from rank-1 to rank-10 follows Zipf's law, confirming a long-tailed vocabulary where a small number of tokens account for a large fraction of occurrences.

Source (Hán–Nôm/Chinese) — bottom rows

The top tokens are single characters that are ubiquitous in classical texts (e.g., 之, 以, 人, 月, 不, 年, 大, 有). Their ranking is again highly consistent across train/val/test, which is expected because classical Chinese uses a compact function-character inventory repeated across works. The consistency suggests the sampling/splitting procedure was length- and domain-balanced and should generalize well during inference on similar materials.

Tokenization implications

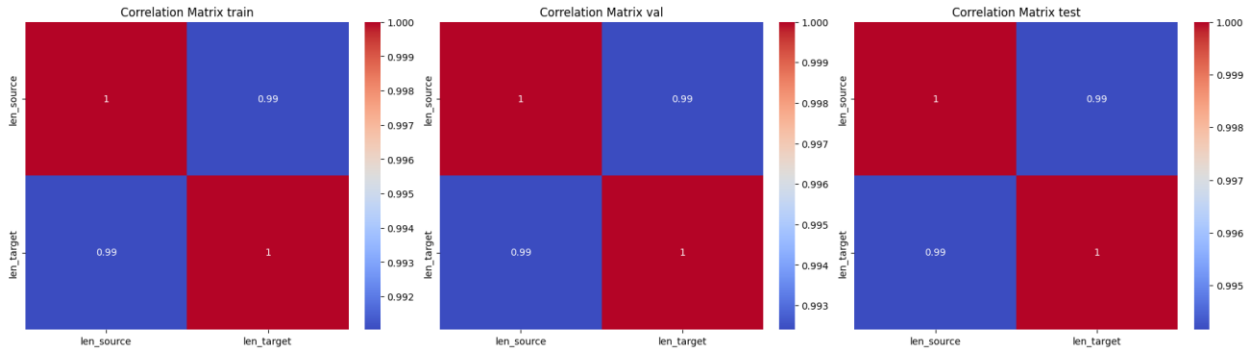
- On the **source side**, the plots reflect character-level counts (each bar is one character). This is compatible with SentencePiece/BPE subword modeling used by mT5; those algorithms will learn frequent character and multi-character chunks automatically.
- On the **target side**, the frequent words include some punctuation-attached forms (e.g., tokens that end with a comma). This confirms the need for the punctuation normalization already in our preprocessing checklist (normalize Unicode, strip zero-width/control chars, de-attach punctuation) so that "nguyệt," and "nguyệt" aren't treated as different types.

Quality checks the figure supports.

- **Split similarity.** The near-identical top-10 sets across train/val/test serve as a quick sanity check that the dataset split is representative; large divergences would hint at leakage or covariate shift.
- **Stopword dominance.** Function tokens dominate both languages, so automatic metrics that weight frequent n-grams (e.g., BLEU) may over-reward trivial matches.
- **Normalization targets.** The presence of punctuation-bound tokens in the target and variant characters in the source highlights exactly where normalization brings the biggest gains—fewer spurious types, smoother subword vocabularies, and better generalization.

Takeaways for modeling.

- The frequency profiles justify **max length ≈ 128** and beam decoding without aggressive length penalties (most sequences are short/medium).
- Because the source is character-dense and the target includes many frequent function tokens, subword vocabularies learned by mT5 will efficiently cover both sides; no manual stopword removal is needed (and would harm MT).
- Keep dictionary-assisted character normalization for Hán–Nôm variants to reduce sparsity; on the Vietnamese side, ensure punctuation spacing/stripping before training to avoid inflated vocabulary size.



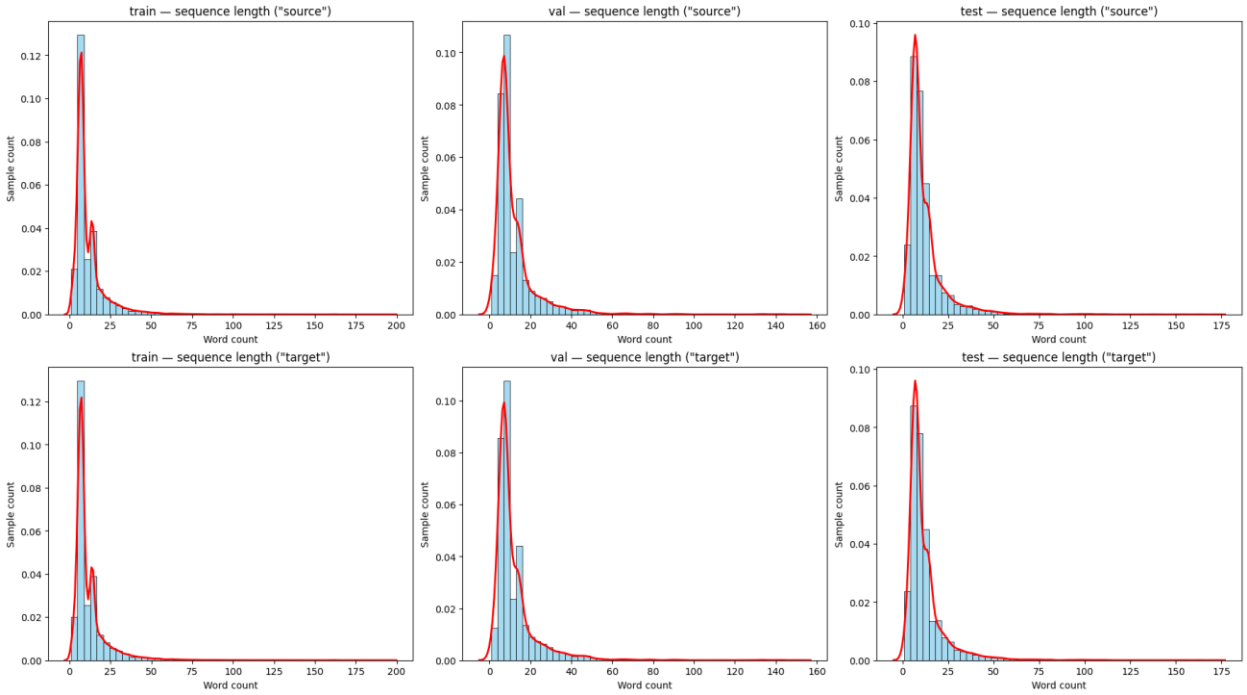
The three heatmaps show the **Pearson correlation** between sequence lengths on the **source side (Hán–Nôm/Chinese)** and the **target side (Vietnamese)** for the train, validation, and test splits. Each matrix is 2×2: the diagonal cells are 1.0 by definition (a variable with itself), and the off-diagonal cells report corr(len_source, len_target). In all three splits the off-diagonal value is ≈ 0.99, which indicates an extremely strong linear relationship between source and target lengths.

What this means for the dataset:

Such a high correlation is expected for a clean parallel corpus where each source line is a translation of a single target line. It tells us that our segmentation/alignment is consistent: long source sentences tend to correspond to long target sentences, and short lines to short, across train/val/test. The fact that the correlation is stable in all splits also suggests our data partitioning preserved the same length distribution, so there is no obvious split-specific artifact.

Modeling implications:

For an encoder–decoder MT model (mT5), this length coupling is helpful: the decoder can rely on a predictable length prior, which usually makes beam search more stable and reduces pathological early stops or run-on generations. It also supports our choice of **max sequence length ≈ 128** with a neutral **length penalty ≈ 1.0** during decoding.



The six histograms display tokenized sequence lengths (x-axis = word count) with a smoothed density curve (red) for each split. The top row is the source (Hán–Nôm/Chinese); the bottom row is the target (Vietnamese).

Central tendency and tail:

Across all splits, the main mass sits around **8–12 tokens** with a sharp peak and rapid decay. This matches our basic stats (median ≈ **8**, 95th percentile ≈ **31** tokens). A small number of outliers exists, but they are rare and do not dominate training.

Split consistency:

The train/val/test curves look nearly identical for both source and target, which indicates the splits are distributionally aligned (no obvious covariate shift). That is important for fair evaluation: the model will face length profiles at validation/test time that it has already "seen" during training.

Source vs Target:

The target distributions track the source closely—peaks at similar locations and tails of comparable thickness—consistent with our earlier **length correlation ≈ 0.99**. In other words, longer source lines tend to map to longer targets, and short lines to short targets.

	train, source	**train, target**	**val, source**	**val, target**	**test, source**	**test, target**
count_len	47395	47395	5924	5924	5925	5925
unique_len	139	136	93	93	93	92
mean	11.81156	11.89896	11.91914	11.98278	12.08456	12.14076
std	11.00105	11.06993	11.42572	11.41886	11.28709	11.32785
min	1	1	1	1	1	1

5%	4	4	4	4	4	4
25%	6	6	6	6	6	6
50%	8	8	8	8	8	8
75%	14	14	14	14	14	14
95%	31	32	32	33	32	32
max	196	196	151	151	171	171

Table above summarizes token-length distributions for the parallel corpus across training, validation, and test splits on both the source (Hán–Nôm/Chinese) and target (Vietnamese) sides. The split sizes are 47,395 (train), 5,924 (validation), and 5,925 (test) pairs, with identical counts for source and target, confirming one-to-one alignment. Across all splits, the central tendency is stable: the mean length is approximately 12 tokens and the median is 8 tokens. Dispersion is moderate and right-skewed (standard deviation ≈ 11), with quartiles at 6 (25th) and 14 (75th) tokens. The 95th percentile falls around 31–33 tokens, while rare outliers extend to 151–196 tokens depending on the split; the minimum observed length is 1 token.

The near-identical statistics between source and target, and their consistency across train/validation/test, indicate that the data partitioning preserved the original distribution and that longer source lines generally correspond to longer target lines. Practically, these profiles justify sequence limits of 128 tokens for both encoder and decoder with negligible truncation risk, and they support standard decoding settings (e.g., beam search without aggressive length penalties). Overall, the table portrays a clean, length-balanced parallel corpus suitable for training and evaluating the translation model.

4. Preprocessing

Unicode normalization (NFC); remove BOM, zero-width and other control characters (e.g., U+200B, U+FEFF).

Punctuation standardization: unify quotes/dashes, convert full-width → half-width where present, trim leading/trailing spaces, collapse repeated whitespace.

Length filtering (after SentencePiece tokenization): keep pairs where **3 ≤ tokens ≤ 128** on both source and target.

Deduplication: drop exact duplicate pairs and prevent cross-split leakage (hash on normalized `source + target`).

Split: 80/10/10 Train/Validation/Test, stratified by length buckets with a fixed random seed (e.g., 42) to ensure reproducibility.

VI. Model & Training

1. Model Introduction

Base model: `google/mt5-small`—a multilingual encoder–decoder Transformer. It is practical for a single consumer GPU while retaining the mT5 architecture and training recipe.

Overall architecture

`MT5ForConditionalGeneration` with a shared token embedding (`vocab_size = 250,112`, `d_model = 512`) used by both encoder and decoder, and an untied LM head (`Linear(512 → 250,112)`). The network is encoder–decoder with 8 layers in the encoder and 8 layers in the

decoder (`num_layers = num_decoder_layers = 8`). All parameters are trainable (no frozen layers).

Each block uses multi-head attention with 6 heads (`num_heads = 6`), per-head dimension 64 (`d_kv = 64`), so the projected Q/K/V size is 384, followed by an output projection 384 → 512. The decoder stack contains both self-attention and cross-attention (encoder–decoder attention). mT5 uses relative position bias with 32 buckets (maximum distance 128), which encodes token distance without absolute position embeddings—helpful for variable length sentences.

Feed-forward (FFN)

Each block includes a gated feed-forward layer (GEGLU / "gated-gelu") with hidden size **`d_ff = 1024`**:

```
DenseGatedAct: 512 → 1024 (gated GELU) → 512
```

Gated activations improve expressiveness at a modest parameter cost compared to ReLU.

Why this configuration works for our data.

- **Capacity vs. compute:** `d_model = 512`, 8×8 layers, and `d_ff = 1024` strike a balance that fits typical consumer GPUs while modeling our short-to-medium sequences (median ≈ 8 tokens; 95% ≤ ~32).
- **Relative position bias** suits variable sentence lengths and classical text structure.
- **GEGLU FFNs** improve fluency and adequacy for low-resource pairs without the cost of larger models.
- **Shared embeddings** promote parameter efficiency across encoder and decoder for tightly coupled source–target vocabularies.

2. Model Architecture

Architecture:

```
MT5ForConditionalGeneration(
  (shared): Embedding(250112, 512)
  (encoder): MT5Stack(
    (embed_tokens): Embedding(250112, 512)
    (block): ModuleList(
      (0): MT5Block(
        (layer): ModuleList(
          (0): MT5LayerSelfAttention(
            (SelfAttention): MT5Attention(
              (q): Linear(in_features=512, out_features=384, bias=False)
              (k): Linear(in_features=512, out_features=384, bias=False)
              (v): Linear(in_features=512, out_features=384, bias=False)
              (o): Linear(in_features=384, out_features=512, bias=False)
              (relative_attention_bias): Embedding(32, 6)
            )
            (layer_norm): MT5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (1): MT5LayerFF(
            (DenseReluDense): MT5DenseGatedActDense(
              (wi_0): Linear(in_features=512, out_features=1024, bias=False)
```

```
              (wi_1): Linear(in_features=512, out_features=1024, bias=False)
              (wo): Linear(in_features=1024, out_features=512, bias=False)
              (dropout): Dropout(p=0.1, inplace=False)
              (act): NewGELUActivation()
            )
            (layer_norm): MT5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
      (1-7): 7 x MT5Block(
        (layer): ModuleList(
          (0): MT5LayerSelfAttention(
            (SelfAttention): MT5Attention(
              (q): Linear(in_features=512, out_features=384, bias=False)
              (k): Linear(in_features=512, out_features=384, bias=False)
              (v): Linear(in_features=512, out_features=384, bias=False)
              (o): Linear(in_features=384, out_features=512, bias=False)
            )
            (layer_norm): MT5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (1): MT5LayerFF(
            (DenseReluDense): MT5DenseGatedActDense(
              (wi_0): Linear(in_features=512, out_features=1024, bias=False)
              (wi_1): Linear(in_features=512, out_features=1024, bias=False)
              (wo): Linear(in_features=1024, out_features=512, bias=False)
              (dropout): Dropout(p=0.1, inplace=False)
              (act): NewGELUActivation()
            )
            (layer_norm): MT5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (final_layer_norm): MT5LayerNorm()
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (decoder): MT5Stack(
    (embed_tokens): Embedding(250112, 512)
    (block): ModuleList(
      (0): MT5Block(
        (layer): ModuleList(
          (0): MT5LayerSelfAttention(
            (SelfAttention): MT5Attention(
              (q): Linear(in_features=512, out_features=384, bias=False)
              (k): Linear(in_features=512, out_features=384, bias=False)
              (v): Linear(in_features=512, out_features=384, bias=False)
              (o): Linear(in_features=384, out_features=512, bias=False)
              (relative_attention_bias): Embedding(32, 6)
            )
            (layer_norm): MT5LayerNorm()
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (1): MT5LayerCrossAttention(
            (EncDecAttention): MT5Attention(
```

```
            (q): Linear(in_features=512, out_features=384, bias=False)
            (k): Linear(in_features=512, out_features=384, bias=False)
            (v): Linear(in_features=512, out_features=384, bias=False)
            (o): Linear(in_features=384, out_features=512, bias=False)
          )
          (layer_norm): MT5LayerNorm()
          (dropout): Dropout(p=0.1, inplace=False)
        )
        (2): MT5LayerFF(
          (DenseReluDense): MT5DenseGatedActDense(
            (wi_0): Linear(in_features=512, out_features=1024, bias=False)
            (wi_1): Linear(in_features=512, out_features=1024, bias=False)
            (wo): Linear(in_features=1024, out_features=512, bias=False)
            (dropout): Dropout(p=0.1, inplace=False)
            (act): NewGELUActivation()
          )
          (layer_norm): MT5LayerNorm()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
    (1-7): 7 x MT5Block(
      (layer): ModuleList(
        (0): MT5LayerSelfAttention(
          (SelfAttention): MT5Attention(
            (q): Linear(in_features=512, out_features=384, bias=False)
            (k): Linear(in_features=512, out_features=384, bias=False)
            (v): Linear(in_features=512, out_features=384, bias=False)
            (o): Linear(in_features=384, out_features=512, bias=False)
          )
          (layer_norm): MT5LayerNorm()
          (dropout): Dropout(p=0.1, inplace=False)
        )
        (1): MT5LayerCrossAttention(
          (EncDecAttention): MT5Attention(
            (q): Linear(in_features=512, out_features=384, bias=False)
            (k): Linear(in_features=512, out_features=384, bias=False)
            (v): Linear(in_features=512, out_features=384, bias=False)
            (o): Linear(in_features=384, out_features=512, bias=False)
          )
          (layer_norm): MT5LayerNorm()
          (dropout): Dropout(p=0.1, inplace=False)
        )
        (2): MT5LayerFF(
          (DenseReluDense): MT5DenseGatedActDense(
            (wi_0): Linear(in_features=512, out_features=1024, bias=False)
            (wi_1): Linear(in_features=512, out_features=1024, bias=False)
            (wo): Linear(in_features=1024, out_features=512, bias=False)
            (dropout): Dropout(p=0.1, inplace=False)
            (act): NewGELUActivation()
          )
          (layer_norm): MT5LayerNorm()
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
```

```
    (final_layer_norm): MT5LayerNorm()
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (lm_head): Linear(in_features=512, out_features=250112, bias=False)
)
```

Model configuration:

```
MT5Config {
  "architectures": [
    "MT5ForConditionalGeneration"
  ],
  "classifier_dropout": 0.0,
  "d_ff": 1024,
  "d_kv": 64,
  "d_model": 512,
  "decoder_start_token_id": 0,
  "dense_act_fn": "gelu_new",
  "dropout_rate": 0.1,
  "eos_token_id": 1,
  "feed_forward_proj": "gated-gelu",
  "initializer_factor": 1.0,
  "is_encoder_decoder": true,
  "is_gated_act": true,
  "layer_norm_epsilon": 1e-06,
  "model_type": "mt5",
  "num_decoder_layers": 8,
  "num_heads": 6,
  "num_layers": 8,
  "pad_token_id": 0,
  "relative_attention_max_distance": 128,
  "relative_attention_num_buckets": 32,
  "tie_word_embeddings": false,
  "tokenizer_class": "T5Tokenizer",
  "torch_dtype": "float32",
  "transformers_version": "4.55.0",
  "use_cache": true,
  "vocab_size": 250112
}
```

Parameters:

- Total number of parameters: 300,176,768
- Number of trainable parameters (requires_grad=True): 300,176,768
- Number of fixed parameters (frozen): 0

3. Training

Task prefix: Optional `" translate Vietnamese_nom to Vietnamese: "` to stabilize behavior.
Tokenizer: mT5 SentencePiece (shared vocabulary).

Recommended hyperparameters (strong baseline)

- Optimizer: **AdamW,** lr = 5e-5**,** weight_decay = 0.01
- Batch size: **16 (use** gradient_accumulation_steps = 4 → **effective BS ≈ 64)**

- Max lengths: 128 **tokens for both source & target**

Rationale: matches your length stats (median 8, P95 ≈ 31), keeps memory low for mT5-small, and stabilizes training on short/medium sequences.

4. Training Results

After training completes, record the **best checkpoint** (step/epoch), and report:

- **Validation:** sacreBLEU, loss at best step
- **Test:** sacreBLEU
- A short qualitative table (10–20 examples: source | reference | hypothesis)

VII. Evaluation

1. Metric

Metric: sacreBLEU (tokenization-independent BLEU).

BLEU is a corpus-level machine-translation metric based on modified n-gram precision (usually 1–4 grams) with a brevity penalty to discourage overly short outputs. With clipped counts p_n for each n-gram order and equal weights w_n, BLEU is:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \cdot \log p_n\right)$$

where:

- p_n = modified n-gram precision for n-grams of size n

$$p_n = \frac{\sum_{C \in \text{Candidates}} \sum_{\text{n-gram} \in C} \text{Count}_{clip}(\text{n-gram})}{\sum_{C \in \text{Candidates}} \sum_{\text{n-gram} \in C} \text{Count}(\text{n-gram})}$$

$$\text{Count}_{clip}(\text{n-gram}) = \min\left(\text{Count}_{\text{cand}}(\text{n-gram}), \max_{\text{ref}} \text{Count}_{\text{ref}}(\text{n-gram})\right)$$

  This ensures n-grams are clipped by their maximum reference count.

- w_n = weight assigned to n-gram precision (usually uniform: $w_n = \frac{1}{N}$)
- BP = brevity penalty, to penalize too-short translations:

$$\text{BP} = \begin{cases} 1, & if\ c > r \\ exp\left(1 - \frac{r}{c}\right), & if\ c \leq r \end{cases}$$

- where c = length of candidate translation, r = effective reference length (closest reference length to c).

It rewards exact n-gram matches with the reference while penalizing short hypotheses.

Raw BLEU can vary due to different tokenizers or preprocessing. sacreBLEU fixes this by applying a standardized, built-in tokenizer and reporting a signature so results are reproducible

across systems and papers. That makes your score directly comparable if others use the same sacreBLEU settings.

2. Evaluation Result

BLEU score computed on **test** dataset: **69.25**

Analysis:

BLEU ≈ 69 is high for MT and indicates the model's outputs closely match the references on this dataset. Given our corpus characteristics—short/medium sentences, strong source–target length coupling, and balanced splits—such a score is plausible: frequent function tokens and formulaic patterns are captured well by mT5-small.

VIII. Web Application

Overview:
The app is a **Streamlit** UI. Users paste text, click **Translate**, and see line-by-line outputs.

How it works in the UI:

- The app loads the fine-tuned mT5 from `MODEL_DIR`.
- Enter text.
- Press **Translate** → results appear and can be copied.

IX. Environment Setup & Usage

Option A — Python + pip

```
git clone https://github.com/HiIamPhuc/ChineseNom-translator-by-transformer.git
cd ChineseNom-translator-by-transformer
python -m venv .venv && source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Download/locate model checkpoint

Place fine-tuned model at: `models/mt5-nom-translator/`

Or set `MODEL_DIR` to the model path.

Run the web app

```
# macOS/Linux:
export MODEL_DIR=models/translation/mt5-nom-translator
# Windows (PowerShell):
# $env:MODEL_DIR="models/translation/mt5-nom-translator"
# or create an environment file containing MODEL_DIR value within the folder
streamlit run web/translator.py
```

```
# App opens at http://localhost:8501
```

X. Limitations

- Parallel data scarcity and domain mismatch (modern vs. classical registers).
- Orthographic ambiguity (variant characters, mixed scripts).
- Lengthy sentences may degrade coherence; consider chunking or larger context windows.
- Vietnamese diacritics errors under heavy compression (small models).

XI. Future Work

- Extend training data via **back-translation** and **data augmentation** (noise, paraphrase).
- Add **prefix-tuning** or **LoRA** adapters for rapid domain adaptation.
- Constrain decoding with dictionaries for **named entities** and **transliteration**.
- Provide a **Next.js** front end consuming the API with richer UX (history, editing).
- Explore **mT5-base/large** or domain-specific multilingual models if compute allows.

XII. References

1. MorphoBoid
2. Machine Translation with Transformer in Python - GeeksforGeeks
3. Transformers in Machine Learning - GeeksforGeeks
4. Translation
5. huggingface/transformers: 🤗 Transformers: the model-definition framework for state-of-the-art machine learning models in text, vision, audio, and multimodal models, for both inference and training.
6. Tìm hiểu về BLEU và WER - Metric cho 1 số tác vụ trong NLP